FTC 2026-27 · BIOBUZZ PRESENTED BY RTX

Robot Master Guide

Takes you from a fresh clone to a tuned, verified robot. Written for someone who can deploy code but is new to robotics. It covers every tuning procedure, every switch and constant in `config.yaml`, and every driver control.

Status at a glance (repo snapshot, 2026-11-05)

- 🔴 **Shot table uncalibrated**: the robot will not feed a ball (§7.6)
- 🔴 **Auto shot windows** are placeholders (§7.7)
- 🟡 **Turret** feedforward off, pivot offset unmeasured (§7.4)
- 🟡 **No ball-count sensor**: the operator stops intake at 4 (§10.3)
- 🔴 **Camera mount** not measured (§7.8)
- 🟡 **Drivetrain (Pedro)** values are provisional (§7.2)
- 🟡 **Flywheel recovery kick** off (§7.5)
- 🟢 **Safety, routes, logic** covered by tests and proofs (§8)

Repository **biobuzz** · FTC SDK 12 · Pedro Pathing 3.0.2 · Java 21 · Gradle
Numbers quoted are from `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/robot/config/config.yaml` on 2026-11-05. If that file has changed, it wins over this guide.
Every yellow or blue link in this PDF is clickable, and the PDF has a bookmark outline.

Contents

1 Read this first 3

The five-minute mental model 3

2 Vocabulary 4

3 Hardware map and ports 5

4 Getting code onto the robot 6

4.1 One-time setup 6

4.2 Connect to the Control Hub 6

4.3 Build and install 6

4.4 Hot-push config, the tuning loop 6

4.5 Getting files off the robot 7

5 How configuration works 7

6 The bring-up order 8

What each tuner writes 8

7 Tuning procedures 9

7.1 Sanity tests 9

7.2 Drivetrain (Pedro AutoTune) 9

7.3 Driver feel and safety 10

7.4 Turret 11

7.5 Flywheel and hood 12

7.6 Shot table 12

7.7 Autonomous shot windows 13

7.8 Vision and HIVE tracking 14

7.9 Poses and routes 14

8 Verification 15

8.1 `verifyBuild` 15

8.2 What to run after a change 15

8.3 On-robot acceptance tests 15

9 Match day 16

9.1 Pre-match checklist 16

9.2 Autonomous routines 16

9.3 Gamepad cheat sheet 17

10 Config reference 18

10.1 pedro 18

10.2 field · hive · match 18

10.3 inventory 19

10.4 turret 19

10.5 vision 20

10.6 sentinel 20

10.7 casablanca 20

10.8 shooter 21

10.9 auto 23

10.10 teleop and poses 23

10.11 Every switch in one table 24

11 Troubleshooting 25

12 Known gaps and further reading 26

1 Read this first

↑ contents

This repository is a complete robot program, and almost none of the robot's behavior is hard-coded in Java. It comes from one YAML file. Making the robot work takes three steps: **install the app**, **run the calibration OpModes** on the real robot (each one prints numbers), and **paste those numbers into** `config.yaml`. Then you run the automated checks and the on-robot acceptance tests.

As of this snapshot the robot is safe but will not score. These values still have to be replaced:

- `shooter.shot_table.calibrated: false` . The shot rows are invented, and **no ball is ever fed** (not in AUTO, TeleOp, or the D-pad Up fixed shot) until you run Ballistics Calibration and set this to `true` .
- `auto.shot_time_points` has 2 placeholder rows. Run Shot Timing Tuner.
- `vision.camera.*` is unmeasured, so HIVE tracking and relocalization will be wrong until it is measured.
- `pedro.*` is carried over from last season. The migration notes call the gains, feedforward and braking *provisional*, and `max_forward_velocity` is flagged "recheck on robot".
- `turret.kv: 0` (no chassis feedforward), `turret.pivot_*_in: 0` (unmeasured), `shooter.kick.wheel_tau_sec: 0` (recovery kick off), `inventory.entry_sensor_enabled: false` (no ball counter).

The golden rules

1. Never edit `config.java` or `config-schema.json` . Both are generated.
2. Numbers go in `config.yaml` , descriptions in `config-docs.yaml` . Then run `./gradlew verifyBuild` .
3. Write poses for **RED** only. BLUE is the letter `m` (RED rotated 180° about the field centre).
4. Never bypass `Sentinel` / `Casablanca` . They keep the robot off the HIVE frame (ramming it is a foul, G417).
5. Test slowly. Anything that spins goes on blocks first.

How to use this PDF

- New to the robot: read §1–6, then follow §6 in order.
- Tuning one subsystem: jump to its card in §7. Every card lists the buttons and exactly which keys to paste.
- Looking up a constant: §10. Every key has a status chip saying whether *you* must measure it.
- At competition: §9 has the checklist and both gamepads.

The five-minute mental model

Drivetrain + odometry
4 mecanum wheels. A goBILDA **Pinpoint** with two odometry pods tracks where the robot is. **Pedro Pathing** follows paths using that position.

Intake
Two motors (front and middle rollers) pull POLLEN in and pass it to the shooter. It caps itself at 4 elements only if a beam break is fitted.

Shooter
Two motors drive one flywheel, and a **hood** servo sets the launch angle. A measured **shot table** maps distance to (hood, flywheel speed).

Turret
A continuous-rotation servo swings the shooter about ±26°. An analog potentiometer reads the angle. It aims at the upward CELL on its own.

Vision
A webcam reads AprilTags under each CELL. Their height tells it which CELL faces up, and so whether the HIVE has TIPPED.

Safety layer
Sentinel knows the no-go zones (JTS polygons). **Casablanca** slows and pushes back driver input near them, and holds heading.

Shot controller
Feeds a ball only when *every gate* passes: flywheel in band, turret aligned, solution valid *and current*, HIVE settled.

Config
One YAML file holds every constant. It can be hot-pushed over ADB in seconds without rebuilding the app.

Proofs + tests
JVM tests simulate turret and flywheel physics and check every auto route. OpenJML and Lean proofs cover the turret and feed logic.

2 Vocabulary

↑ contents

The game

Term	Meaning
POLLEN	2.8 in yellow balls. This robot's only game piece.
NECTAR	3.6 in alliance-colored balls. This robot cannot handle them.
SCORING ELEMENT	Either kind of ball.
HIVE	The pivoting scoring structure. There is one per alliance, near the field centre.
CELL A / B	The two openings of a HIVE. You may only launch into the one facing *up*. **A** starts up (RED: the audience-side CELL. BLUE: the rear CELL).
TIP	Enough elements tip the HIVE (20 pts), which turns the other CELL upward. Launching while it swings is not allowed.
FLOWER	Wall-mounted POLLEN dispenser. This robot has no FLOWER mechanism.
LOADING ZONE	Taped zone on the alliance wall. Ending AUTO partly inside it, off the wall, scores PARK (and LEAVE).
G402, G407, G417…	Manual rule numbers the code enforces: G402 stay on your side in AUTO, G407 at most 4 elements, G417 upward CELL only and no ramming the HIVE.
AUTO / TELEOP	30 s autonomous, then 120 s driven.

The robot

Term	Meaning
OpMode	A program picked on the Driver Station. Groups: *Tuning*, *Calibration*, *Test*.
Control Hub / DS	The robot's computer, and the phone plus gamepads that run OpModes and show **telemetry** (on-screen text).
Panels	A web dashboard (bylazar Panels). Some tuners draw the field on it and expose live-editable fields there.
Pose	`[x in, y in, heading °]` on the field.
Pedro frame	Origin at the audience-wall/RED-wall corner. +x toward BLUE, +y away from the audience. Heading 0 = +x, counter-clockwise positive. The field is 141.5 in, centre (70.75, 70.75).
P, I, D	Feedback gains. P pushes in proportion to the error, I removes a steady offset, D damps. Too much P makes it oscillate.
kS, kV (feedforward)	Power applied *before* any error appears. kS breaks static friction, kV is power per unit of speed.
Ticks/s ("rpm")	Every `*_rpm` key in `shooter` is really encoder ticks per second.
Slew limit	A cap on how fast drive power may change, so the odometry pods stay on the floor.
`m`	A BLUE value meaning "RED rotated 180° about the centre".

3 Hardware map and ports

↑ contents

Create the Robot Configuration on the Driver Station with **exactly** these names (they are case-sensitive). The ports come from `TeamCode/.../teamcode/hwmap.yaml`. A wrong name makes the OpMode stop at INIT with "device not found".

Hub · port	Name	Type	Purpose / notes
Control · motor 0–3	`leftFront` `rightFront` `leftBack` `rightBack`	DC motor	Mecanum drive, in that port order. Directions are set and confirmed by Pedro's Mecanum Tuner (§7.2).
Control · I²C 0	`pinpoint`	goBILDA Pinpoint Driver	Odometry computer. It is also the preferred tilt sensor for the Pod Contact Tuner.
Control · digital 0	`intakeBeam`	Digital channel	*Optional* intake-entry beam break that counts balls (G407). Also set `inventory.entry_sensor_enabled: true`.
Expansion · motor 0, 1	`intakeFront` `intakeMid`	DC motor	Intake rollers.
Expansion · motor 2, 3	`shooter2` `shooter`	DC motor + encoder	One flywheel driven from both sides: `shooter` runs FORWARD and `shooter2` REVERSE. Speed is read from the encoder.
Expansion · servo 3	`hood`	Servo	Launch angle.
Expansion · servo 4	`turn`	CR servo	Turret rotation.
Expansion · servo 5	`gate`	Servo	Listed in `hwmap.yaml`, but **no current code uses it**.
Expansion · analog 0	`turnanalog`	Analog input	Turret potentiometer (absolute angle).
Expansion · I²C 1	`turnImu`	IMU	Only for Turret Angle Verification, as ground truth. Its mounting is `turret.orientation.logo/usb`.
USB	`Webcam 1`	Webcam	AprilTag camera, **pitched up** toward the CELLS. The name must match `vision.camera_name`.
(any servo port)	`light`	Servo (goBILDA RGB)	Used only by *Lumos Test*. The match code does not drive it, so it is optional.
(fallback)	`imu`	IMU	Tilt fallback for the Pod Contact Tuner when the Pinpoint firmware lacks pitch/roll (`casablanca.jump_tuner.imu_name`).

Switch: no turret. If the final robot has no turret, set `turret.enabled: false`. `turn` and `turnanalog` become optional, and the shot gate checks chassis heading instead of turret alignment.

4 Getting code onto the robot

↑ contents

4.1 One-time setup

1. Install **Android Studio** and a **JDK 21** (the build targets Java 21).
2. Open the repo root (the folder containing `gradlew`) and let Gradle sync. The robot code is the `TeamCode` module.
3. Make sure `adb` exists. The build looks in `$ANDROID_HOME`, `$ANDROID_SDK_ROOT`, then `~/Library/Android/sdk/platform-tools`.
4. *Optional (proofs):* install Lean with `brew install elan-init`. The first `verifyBuild` then runs `setupOpenJml` and `setupLean`, which download OpenJML and about 5 GB of Mathlib once.

4.2 Connect to the Control Hub

Wi-Fi (default). Join the robot's Wi-Fi. The hub is at `192.168.43.1:5555`.

```
./gradlew connectAdb
./gradlew connectAdb -
PadbAddress=192.168.43.1:5555  # explicit
adb devices          # lists the hub
```

USB. Use a **USB-A to USB-C** cable (REV's standard). On this team's MacBook a C-to-C cable never came up, because the hub took the host role. Use an A-to-C cable or a C-to-A adapter.

> **macOS:** "No route to host" while you are on the robot Wi-Fi usually means macOS is blocking local-network access for your terminal. Allow it in *System Settings › Privacy & Security › Local Network*.

4.3 Build and install

1. Prove the code is healthy, no robot needed:

   ```
   ./gradlew verifyBuild
   ```

   This formats, compiles, runs every unit test, checks `red_*` / `blue_*` symmetry and runs the proofs. It must pass (§8.1).
2. Install: in Android Studio pick the **TeamCode** run configuration, choose the hub as the device, and press ▶ Run. The CLI equivalent is the standard Android task `./gradlew :TeamCode:installDebug`.
3. On the DS you should see *TeleOp*, the six autos (*Red/Blue A: tip first*, *Red/Blue B: second CELL*, *Red/Blue Park Only*) and the tuning OpModes.

> **Gradle prints nothing for over a minute?** The daemon is probably frozen (`ps` shows state `T`). Re-run with `./gradlew --no-daemon <task>`.

4.4 Hot-push config: the tuning loop

While tuning, you change numbers dozens of times. Push the config file instead of rebuilding the app:

Edit `config.yaml` › **Push** `./gradlew pushConfig` › **Re-INIT** stop, then INIT the OpMode again › **Test** watch telemetry › **Lock in** commit, `verifyBuild`, reinstall, `resetConfig`

Command	What it does
`./gradlew pushConfig`	First runs `checkPushConfig` (autonomous-route safety, the shipped-config `ConfigSanity` check and match windows, under ½ s), then copies the file to `/sdcard/FIRST/teamcode/config.yaml` over ADB. A value-only change recompiles nothing. **Re-INIT the OpMode to load it.**
`./gradlew resetConfig`	Deletes that override file, so the robot reverts to the config bundled in the APK.
`./gradlew generateConfig`	Regenerates `config.java` and `config-schema.json` after you add a key. `verifyBuild` runs it too.

> The robot uses a pushed file only if its `meta.season` is `"biobuzz"`. Otherwise the DS shows why it was ignored. A plain `adb push` skips the safety check, and **the robot never re-checks autonomous routes itself**. Before competition, copy the final values into the repo, run `verifyBuild`, reinstall, then `resetConfig`, so there is exactly one source of truth.

4.5 Getting files off the robot

Calibration OpModes write their results to `/sdcard/FIRST/teamcode/` on the hub. Pull them into `calibration-data/` to keep a record. Each file is overwritten on the next run, so pull it before re-running.

```
adb pull /sdcard/FIRST/teamcode/shot_table_pollen.yaml calibration-data/
adb pull /sdcard/FIRST/teamcode/shot_time_table.yaml   calibration-data/
adb pull /sdcard/FIRST/teamcode/shot_timing_log.csv    calibration-data/
adb pull /sdcard/FIRST/teamcode/shooter_characterization.json calibration-data/
```

5 How configuration works

↑ contents

- **One file**, `robot/config/config.yaml`, holds every tunable. `config-docs.yaml` holds each key's description and its `min` / `max`, keyed by the leaf name (e.g. `kv`).
- Java reads values through a generated typed class, for example `config.turret.pidf.p`. No numbers are hard-coded.
- **Adding a key:** add it to the YAML, add a `desc` (and `min` / `max`) to the docs file, then run `generateConfig`. A value outside min/max fails the build.
- **Alliances:** a `red_*` key without a `blue_*` twin fails `checkConfigKeys`. Poses are RED plus `m`. If BLUE measures differently, replace that `m` with explicit numbers.
- **Season tag** `meta.season: "biobuzz"` stays fixed for the whole season. It makes the robot ignore a leftover pushed config from another game.
- **At INIT**, every tuning OpMode shows config problems on the DS: a rejected push, a broken shot table, a bad encoder range, and so on. Read the top of the telemetry.

Status chips used in §10

MEASURE	You must calibrate this on *your* physical robot.
PLACEHOLDER	An invented stand-in that must be replaced before it means anything.
TUNED	Already measured or chosen by simulation. Leave it unless the hardware changes.
POLICY	A safety, feel or strategy choice. Change it deliberately.
SWITCH	A boolean that turns a feature on or off. All of them are listed in §10.11.
FIELD	Game geometry from the manual or CAD. Do not change it.
MODEL ONLY	Read by the physics model and calibration display only. It does not change where a live shot goes.

```yaml
# the shape of a pose (RED), and its BLUE mirror
teleop:
  poses:
    red:
      start: [58.0, 9.0, 90]   # x, y, heading
    blue:
      start: m    # [83.5, 132.5, 270]
```

6 The bring-up order

↑ contents

Each stage depends on the ones before it: the turret can't be tuned on a robot that doesn't know where it is, and the shot table can't be calibrated with a flywheel that doesn't hold speed. Work left to right, top to bottom.

Build + install 0
`verifyBuild` green. The hardware names match. Every OpMode appears on the DS.
§3-4 ›

Sanity tests 1
Each motor, servo and sensor responds and moves the right way.
§7.1 ›

Pedro AutoTune 2
Motor directions, Pinpoint, Foresight. The robot drives out and returns to its start.
§7.2 ›

Driver feel + safety 3
Friction, pod contact, heading lock. Casablanca stops the robot short of the HIVE.
§7.3 ›

Turret 4
Center, plant, verification. It reaches and holds an angle without hunting.
§7.4 ›

Flywheel + hood 5
Characterize the wheel and find the hood's ends. Speed holds and recovers after a ball.
§7.5 ›

Shot table 6
Ballistics Calibration, then `calibrated: true`. Balls drop in from 24-48 in.
§7.6 ›

Shot windows 7
Shot Timing Tuner fills `auto.shot_time_points`.
§7.7 ›

Vision 8
Measure the camera mount. The HIVE readout flips when the HIVE is tipped by hand.
§7.8 ›

Poses + autos 9
Start Pose Finder, then every AUTO at low speed on a real field.
§7.9 ›

Verify + lock in 10
Final values in the repo, `verifyBuild`, reinstall, `resetConfig`.
§8 ›

Rehearse 11
Pass the acceptance tests, then two clean full matches using the checklist.
§9 ›

What each tuner writes

OpMode (DS group)	Writes these keys	Card
Pedro AutoTune (web page)	`pedro.*`: motor directions, pod offsets, heading P, braking, max speeds, decelerations	7.2
Friction Calibration TeleOp (Calibration)	`casablanca.friction.x/y/rot`, `casablanca.heading_lock.ks_moving`, `sentinel.rotation_lookahead_time`	7.3
Casablanca Pod Contact Tuner (Calibration)	`casablanca.smoothing.<axis>.increase_time / decrease_time`	7.3
TeleOp Heading Lock Autotuner (Calibration)	`casablanca.heading_lock.pid.p / d`	7.3
Turret Center Calibration (Tuning)	`turret.travel.*`, `turret.analog_encoder.*`	7.4
Turret Plant Calibration (Tuning)	`turret.ks_positive/ks_negative`, `turret.kv`, `turret.pidf` (p, with i = d = f = 0)	7.4
Shooter Characterization (Calibration)	`shooter.ks`, `shooter.pidf.p/i/d/f`, `shooter.kick.wheel_tau_sec/sensor_lag_sec`	7.5
Hood Tuner (Tuning)	`shooter.ballistics.min_hood_servo_pos / max_hood_servo_pos` (you read them off the screen)	7.5
Ballistics Calibration (Calibration)	`shooter.shot_table.pollen_points`, then set `calibrated: true`	7.6
Shot Timing Tuner (Calibration)	`auto.shot_time_points`	7.7
Tape measure and protractor	`vision.camera.*`, `turret.pivot_*_in`, `sentinel.robot_width`, `shooter.ballistics.launch_height_inches`	7.8
Start Pose Finder (Calibration)	Nothing. It checks that each configured start pose is physically right.	7.9

7 Tuning procedures

↑ contents

Before every session: a charged battery (note its voltage), the robot on blocks for anything that spins, clear floor around it, and a second person on the power switch. Every calibration OpMode reloads config at INIT and reports problems there, so after each `pushConfig`, stop and re-INIT. Many tuners also have live-editable fields in the **Panels** dashboard. Those are noted as Panels.

7.1 Sanity tests

DS group: Test

OpMode	Controls	What to check
Servo Test	RT forward · LT reverse	The turret servo turns both ways. Power is scaled by `turret.max_power_output`, so it never slams the stops.
Turn Analog Test	RT / LT move servo · A reset min/max	The `turnanalog` voltage changes smoothly as the turret moves. It shows the raw volts and the observed min/max, which should stay inside 0–3.3 V with no jumps.
Manual Turret Turn Tester	D↓ zero here · D→ go to `turn_degrees` Panels	Closed-loop turning works. Zero it while centred. It refuses to drive if zeroing off-centre would put a stop out of reach. If the turret moves *away* from its target, flip `turret.servo_direction_inverted`.
Shooter Max Velocity Test	X stop · A restart · Y reset stats	Both flywheel motors spin *opposite* ways (they drive one wheel from both sides). It shows peak speed and time to 95% of target, using the SDK default velocity PID. Sanity-check `shooter.max_rpm` (1500).
Lumos Test	none	The RGB light turns red (a wiring check only).
Clock Test	none	Shows the Control Hub's date and time at INIT, to check the hub clock that stamps files and logs.

Full Max Velocity Test, *Shooter Autotune* and *AtanAlignTest* are `@Disabled` leftovers and do not appear on the DS.

7.2 Drivetrain: Pedro AutoTune

writes pedro.*

Pedro AutoTune procedures

BROWSER PAGE ON THE HUB

needs 4+ ft clear · wheels on floor

STEPS

1. Connect a laptop to the robot's Wi-Fi and open `http://192.168.43.1:10158`.
2. **Mecanum Tuner** spins each drive motor in turn, and you enter whether it went forward or reversed. This sets the motor directions.
3. **Pinpoint Tuner**: pick the pod type, then **push the robot forward exactly 48 in** (encoder scale), then push it forward and then left (pod directions).
4. **Foresight Tuner**: drives itself (48 in by default, and at least 15 in for braking) to measure heading kP, braking coefficients, top speeds and natural deceleration.
5. **Tests**: Hold, Line, Curve, Interpolation, Localization, Odometry, Driving, Pose. Run Localization and Line first.
6. Copy the reported numbers into `config.yaml` under `pedro:` (§10.1), then `pushConfig`. Each procedure reloads config, so no restart is needed.

THEN VERIFY

- **Forward Left Return Test** (Autonomous, group Tests): from where it stands, it drives 20 in forward, 10 in left, then back. It should end where it started. The DS shows the live pose.
- Push the robot by hand at heading 0. Forward must raise x, left must raise y, and turning counter-clockwise must raise the heading.

WATCH OUT

- `max_forward_velocity: 79.97` is flagged in the file as unusually low.
- After a chassis change, redo all four procedures, then §7.3, because Casablanca reuses `pedro.braking`.

7.3 Driver feel and safety (Casablanca)

writes casablanca.*

Friction Calibration TeleOp CALIBRATION

open carpet · ~4 ft clear · Panels thresholds

WHAT IT DOES

It slowly ramps power until the robot actually moves (above 1 in/s, or 0.05 rad/s for rotation, held for 5 loops). That gives the static-friction power on each axis. The rotation test also measures friction while already moving and the top rotation rate.

CONTROLS

- A forward test → kS X
- B strafe test → kS Y
- X rotation test → static, moving, max ω

```
casablanca.friction.x ← X · .y ← Y · .rot ← Rot Static
casablanca.heading_lock.ks_moving ← Rot Moving (keep the old value if it says NOT MEASURED)
sentinel.rotation_lookahead_time ← "Suggested rotation_lookahead_time" (0.45 / max ω)
```

Casablanca Pod Contact Tuner CALIBRATION

one axis per run · 48 in lane both ways

WHY

The odometry pods ride on small spring mounts. Hard acceleration or braking can lift a pod off the floor and corrupt the robot's position. This finds the gentlest power slew limits that keep both pods in contact.

STEPS

1. Set `casablanca.jump_tuner.trial_axis` to `FORWARD`, `STRAFE` or `TURN`, then push and INIT.
2. Place the robot at `jump_tuner.calibration_pose` = **[20, 72, 90]**, a lane beside the HIVE. STRAFE trials start there turned 90°.
3. **Hold** A for each trial: baseline, accelerate, hold, stop, settle. There is no automatic return, so reposition by hand between trials.
4. Repeat until it shows a recommendation. Do the other axes the same way.

CONTROLS

- A hold = run trial (keep holding through the stop)
- X mark "pod lifted" (you saw it lift)
- release A / B / STOP = abort now

OUTPUT

```
casablanca.smoothing.<axis>.increase_time
casablanca.smoothing.<axis>.decrease_time
```

It aborts on more than 10° of tilt, a stale IMU, a stall, or drift. All limits are in `casablanca.jump_tuner.*`.

TeleOp Heading Lock Autotuner CALIBRATION

turns in place

STEPS

1. Place the robot at `heading_tuner.calibration_pose` = **[20, 72, 90]** with room to turn.
2. Press START, then **hold** A for the whole search. It steps the heading ±10° for every (P, D) pair: P from 1 to 10 in 4 steps, D from 0 to 0.9 in 4 steps.
3. It keeps the pair with the best *worst-case* settling time that passes in both directions.

CONTROLS

- A hold = search · release A or press B = abort

OUTPUT

```
casablanca.heading_lock.pid.p ← p
casablanca.heading_lock.pid.d ← d (i stays 0)
```

`heading_tolerance_degrees` must be at least `heading_lock.error_deadband_deg`, or it refuses to start.

Casablanca Test TEST · VERIFY

drives as BLUE | Panels field view

This is normal TeleOp driving through Casablanca (cubic sticks, slow-down, repulsion, heading lock), with live telemetry. **It drives as BLUE** and starts at the BLUE image of the tuner pose, **(121.5, 69.5, 270°)**, so place the robot there. Drive at the HIVE frame from the front, side and diagonals at full speed. It must start slowing about 8 in out (`slow_down`), cut power toward the frame about 2.5 in out (`hard_stop`), and never touch.

PANELS TOGGLES

`rawDirectDrive`: bypass Casablanca to compare
`fieldcentric`: field-relative sticks

IF IT FAILS

Creeps in: raise `hard_stop`, check `sentinel.robot_width` and `pedro.braking`. Feels sluggish: lower `slow_down`. Keep depth and side symmetric.

7.4 Turret

writes turret.* · do the cards in order

① Turret Center Calibration TUNING

servo unpowered | by hand | Panels margin

STEPS

1. Turn the turret **by hand** to the positive hard stop and press A.
2. Turn it to the negative hard stop and press A.
3. Turn it to *roughly* the middle and press A. This only tells it which of the potentiometer's two arcs the turret travels. The exact centre is computed from the stops.
4. B starts over. Paste the printed block.

The travel limits are half the measured span minus `limitSafetyMarginDeg` (3°) Panels. It keeps the existing `degrees_per_volt` unless you set `knownTotalTravelDeg` Panels to re-derive it.

OUTPUT (PASTE)

```
turret:
  travel: { min_angle, max_angle }
  analog_encoder:
    zero_voltage, degrees_per_volt,
    min_voltage, max_voltage,
    full_scale_voltage
```

② Turret Plant Calibration TUNING

turret moves itself | Panels τ

WHY NOT NORMAL AUTOTUNING

A CR servo is an *integrator*: power sets speed, not position, so relay autotuning gives meaningless gains. This OpMode measures the breakaway power in each direction (kS) and the speed per unit of power (K). Then it picks `p = 1/(K·τ)` for a closed-loop time constant τ = `desiredTimeConstantSec` (0.20 s) Panels.

STEPS

1. Press START. The turret ramps power on its own within its travel limits, up to `max_power_output`. B aborts at any time.
2. If it reports **CONTROL LOOP IS INVERTED**, flip `turret.servo_direction_inverted` and re-run before using any closed-loop OpMode.
3. Paste the block.

OUTPUT (PASTE)

```
turret:
  ks_positive: …  ks_negative: …
  kv: … # 1/K, enables feedforward
  pidf: { p: …, i: 0, d: 0, f: 0 }
```

> The convergence proof assumes **i = d = f = 0**, and the repo currently has `i: 0.008`. After *any* turret gain change, run `python3 TeamCode/verification/TurretProof/conditions.py` It must exit 0, and it prints the target distances the proof covers.

③ Turret Angle Verification TUNING

needs turnImu

Compares the encoder angle against an independent IMU mounted on the turret (`turnImu`, oriented by `turret.orientation.logo/usb`). After each move, the encoder and IMU angles should agree within about 1°. A consistent scale error means `degrees_per_volt` is off. It also warns at INIT if a pushed `zero_voltage` would put the encoder's wrap point inside the travel.

THEN MEASURE BY HAND

- `turret.pivot_forward_in`, `pivot_left_in`: the turret axis position from the robot centre. A 4 in error is a 6–9° aim error at BIOBUZZ ranges.
- The lopsided kS (+0.0009 / −0.0663) suggests the servo's neutral is about 0.033 off centre. Re-centre it with a servo programmer, then re-run ②.

CONTROLS

D↑/D↓ target ±5° · D←/D→ ±15°
Y 0° · X +30° · B −30° (clamped to travel)
A zero IMU reference
LT kill power / idle · RB clear safety fault

7.5 Flywheel and hood

writes shooter.*

Shooter Characterization CALIBRATION

~30 s | flywheel clear of balls | battery under load

STEPS

1. Press START, then A. It holds a staircase of open-loop powers, waits for the wheel to settle at each, and least-squares fits `power = kV·speed + kS`. Each point is corrected from the measured battery voltage to `shooter.nominal_voltage`.
2. Then it steps the power from 0.40 to 0.85 and fits the wheel's time constant and the speed sensor's delay.
3. Check the fit's RMS and worst error, which it shows on screen, then paste. A copy is saved as `shooter_characterization.json`.

If `use_ftc_pid` is true, the screen warns you that these values are ignored. `p = 2·kV` (loop gain 2) and `i = p / 2 s` were chosen by simulation sweep and are checked by `FlywheelClosedLoopProperties`.

OUTPUT (PASTE UNDER `SHOOTER:`)

```
ks: …
pidf: { p: …, i: …, d: 0, f: … }
kick: # only if the step test passed
  wheel_tau_sec: …
  sensor_lag_sec: …
```

A non-zero `wheel_tau_sec` turns on the **recovery kick**. After a ball (or during spin-up), it applies as much power as the wheel model allows without overshooting, until the speed is predicted to reach the feed band. If the step test fails, it says so. Leave `kick` alone in that case.

Shooter Velocity Tuner TUNING

Panels

Runs the real `Shooter` controller at `targetRPM` (starts at `constant_rpm`, editable in Panels). It shows speed, error, and the feedforward, P and I terms. Use it to watch spin-up and the dip and recovery when you hand-feed a ball.

Hood Tuner TUNING

D↑/D↓ ±0.01 · D→/D← ±0.001 · A go to `min_hood_servo_pos` · B go to `max_hood_servo_pos`. Walk to each physical end without straining the linkage, then copy the values into those two keys. **The linkage is reversed**: the flattest arc, `min_hood_servo_pos` (0.85), is numerically *larger* than the steepest, `max_hood_servo_pos` (0.0). That is correct.

7.6 Shot table: Ballistics Calibration

writes shooter.shot_table

Ballistics Calibration CALIBRATION

real field + HIVE | RED CELL A up | many POLLEN

PREREQUISITES

Finish §7.2–7.5 first: odometry, turret and flywheel must all behave. Measure `turret.pivot_*_in` first too, because endpoints place the

CONTROLS (GAMEPAD 1)

Sticks drive (through Casablanca)
RT aim + shoot at the pinned speed

turret axis on the ray.

STEPS

1. Place the robot at `teleop.poses.red.shoot_a` **[58, 27, 90]** with RED's audience CELL (CELL A) **facing up**. Leave the ball type on POLLEN (RB toggles it, but NECTAR is disabled).
2. It walks the endpoints **24, 30, 36, 42, 48 in** along a ray 40° off the opening's axis (toward the RED wall). Endpoints beyond the ray's reach (about 52 in) or with an unsafe drive are skipped automatically.
3. For each endpoint it prints the target pose. A confirms, and the robot drives there by itself. B skips it.
4. Load balls and shoot with RT. Adjust the hood and flywheel until shots drop in *consistently*, then press A to accept (or B to skip). It records the *actual* distance from the turret axis.
5. Progress is saved after every endpoint to `/sdcard/FIRST/teamcode/shot_table_pollen.yaml`. **Pull it** (§4.5) before re-running.
6. Replace **only** the `pollen_points` entry with the printed rows (pasting a whole `shot_table:` block makes a duplicate key). Then set `calibrated: true` and push.

RB manual intake/feed · LB/LT stop
X toggle flywheel rev
D↑/D↓ hood coarse ±0.05
D→/D← hood fine ±0.005
L3/R3 (stick clicks) speed −/+ 50
Y reset hood + speed to the seed
A accept · B skip endpoint

OUTPUT

```
pollen_points: [
  24.3, 0.0000, 805, # dist, hood, ticks/s
  …
]
calibrated: true
```

Three things that bite after calibrating.

1. **Range.** A shot is valid only inside the table's distances *and* inside `min_valid_distance` – `max_valid_distance` (24–72). With a 24–48 in table, the robot will not shoot from 50 in. To shoot farther, calibrate along a longer ray.
2. **Speed clamp.** At runtime the table speed is clamped into `preferred_shot_rpm` – `max_shot_rpm` (800–1450). If you accepted rows below 800, the screen notes it. Lower `preferred_shot_rpm` to match, or those shots go hot.
3. **Old data.** The files already in `calibration-data/` are from **last season** (distances up to 124 in). Never paste them.

7.7 Autonomous shot windows: Shot Timing Tuner

writes auto.shot_time_points

Shot Timing Tuner CALIBRATION

robot does NOT drive | fires real balls | needs calibrated table

WHAT IT DOES

For each distance it measures how long a whole volley of `auto.balls_per_volley_count` (4) balls takes, counting balls from the flywheel's speed dips. The robot stays put, but its pose is *faked* to each distance on the same ray Ballistics Calibration uses (clamped into your table), and the turret still slews.

STEPS

1. Load 4 balls (X runs the intake). Wait for the flywheel to read **READY**, then RT fires the volley.
2. Check that the screen counted every ball. If not, adjust the dip trigger with the stick clicks. Then A accepts or B skips.
3. At the end, paste the rows under `auto:`.

CONTROLS

RT fire · LT stop everything
X intake on/off
A accept · B skip
D↑/D↓ expected ball count
L3/R3 dip trigger −/+ 0.5%

OUTPUT

```
auto:
  shot_time_points: [ 24.3, 2350, … ]
# also shot_timing_log.csv (per-loop trace)
```

The ball detector's `dip_fraction` (0.09) must sit between a real ball's dip (12–26% measured) and the deepest non-ball dip (5.4%).

7.8 Vision and HIVE tracking

writes vision.*

1. Mount the webcam so it sees the CELLS. Their tags are 30–60 in above the floor, so tilt it up. The default tilt is 50°.
2. **Measure** and enter `vision.camera.forward_in` and `left_in` (from the robot centre), `height_in` (lens above the floor), `yaw_deg` and `pitch_up_deg`. The current values (3.54, 4.72, 7.68, 0, 50) were never measured.
3. Check that `vision.camera_name` matches the DS configuration (`Webcam 1`).
4. In TeleOp, tip the HIVE by hand. The HIVE telemetry must flip after `hive.confirm_frames_count` (3) agreeing frames.
5. Relocalization: with the HIVE settled and within 72 in, hold G2 D↓. The pose should snap toward the truth. It is rejected if the jump exceeds 18 in (`relocalize.max_correction_in`).

Also measure with a tape

`sentinel.robot_width`	The robot's outer width (it is modelled as a square). Every safety zone depends on it.
`turret.pivot_forward_in / pivot_left_in`	Turret axis offset from the robot centre.
`shooter.ballistics.launch_height_inches`	Ball release height above the tiles (physics model).
`pedro.localization.x/y_pod_offset`	Pod positions, which the Pinpoint Tuner then confirms.

Without a working camera, set `vision.enabled: false`. The operator then records TIPS with G2 LB.

7.9 Poses and autonomous routes

writes poses

Start Pose Finder CALIBRATION

1. **In INIT:** D← / D→ picks the alliance and D↑ / D↓ picks the CELL A or CELL B routine. It shows that routine's configured start pose.
2. Set the robot's **centre on the tile-seam crossing two tiles from the audience wall and two from the RED wall**, facing +x (toward BLUE). Odometry starts there at (47.17, 47.17, 0°).
3. Press START. Drive with the sticks (slow, robot-centric), following "Move ±x, ±y" until the distance to start is about 0.
4. Check the robot sits legally (touching the wall, on your side, not in the LOADING ZONE, not touching a FLOWER). Press A to re-zero odometry there.

THEN DRIVE EVERY AUTO SLOWLY ON A REAL FIELD

- Adjust **RED** poses only. BLUE follows through `m`.
- Paths must keep the robot's swept footprint on your side and clear of the HIVE frame. `AutoRouteSafetyTest` enforces this in `verifyBuild` and every `pushConfig`.
- Every routine must end parked: partly in the LOADING ZONE and off the wall.
- `score_b_cp` and `alt_via` are path control points and waypoints. Move them to bend a path away from the HIVE.

8 Verification

↑ contents

8.1 The one command: `./gradlew verifyBuild`

Step	What it protects
`generateConfig`	Rebuilds the typed config from YAML. It fails on a value outside `min` / `max` or a key without a description.
`spotlessApply`	Auto-formats the Java.
`testDebugUnitTest`	Every JUnit and property test on your computer, no robot needed. Highlights: **AutoRouteSafetyTest** checks every autonomous path for both alliances against the HIVE frame, the centre line, the walls and LEAVE/PARK. **TurretClosedLoopProperties** runs the real turret code against a simulated plant (random gain, stiction, lag, noise, jitter). **FlywheelClosedLoopProperties** runs the real shooter against a simulated flywheel (kS ±30%, kV ±10%, 11.5–13.8 V, sensor lag, ball hits). Plus Sentinel/Casablanca symmetry, HIVE tracker, match clock, inventory, shot solver and table, config loader and season guard, and **TunerPoseSafetyTest**.
`checkConfigKeys`	Fails if a `red_*` key lacks its `blue_*` twin.
`verifyTurret`	OpenJML proves the turret control law, the feed gate (G417, solution currency), the HIVE tracker's transitions, the anti-windup integrator and the ball inventory. `-PexactDoubles` proves with bit-exact IEEE-754 rounding (minutes).
`verifyTurretLean`	A Lean 4 proof that the turret, from settled or approaching a still target, ends settled and aimed, for any plant in the stated box and gains meeting the conditions.

The proofs and tests are cached on their inputs, so a value-only change re-runs quickly. The proofs also run with every assemble/install.

8.2 What to run after a change

You changed…	Run
Any config number while tuning	`pushConfig` (it checks itself), then re-INIT
Turret gains (`pidf` , `ks_*` , `kv`)	`conditions.py` (must exit 0), then `verifyBuild`
Flywheel kS/kV/gains/kick/voltage	`verifyBuild` . Flywheel properties must pass.
Route poses, field geometry, `robot_width`	`verifyBuild` (route safety). **Mandatory.**
Added a config key	`generateConfig` , then `verifyBuild`
Any Java	`verifyBuild`

> A rejected push names the failing test. The usual causes are a route crossing the centre line or touching the HIVE frame, shot-table distances that don't strictly increase, a bad encoder range, or a season mismatch.

8.3 On-robot acceptance tests

recommended · log date + battery V

#	Test	Pass
1	Forward Left Return Test	Back within ~1 in
2	Full speed at the HIVE (front/side/diagonal)	Stops short, never touches
3	Release the turn stick after turning	Holds heading, no hunting
4	±20° steps in Angle Verification	<1 s settle, encoder ≈ IMU
5	G2 RT at each table distance	≥4/5 in, at every row
6	4-ball volley	All counted, no jam
7	Aim while driving under 20 in/s	Tracks; refused above limit
8	Tip the HIVE by hand mid-shot	Stops, flips, 1.5 s lockout
9	Intake a 5th ball	Stops at 4 (or operator does)
10	Each AUTO from its start	Parked in 30 s, no contact
11	TeleOp to 0:20 left	Park alert fires
12	AUTO then TeleOp	State inherited (<2 min)
13	Low battery (about 11.5 V)	Still reaches feed band

9 Match day

↑ contents

9.1 Pre-match checklist

1. Charged battery. Connect the hub and the DS. INIT shows **no config warning** (ignored override, shot-table error, uncalibrated table).
2. Agree on autos with your partner: one *A: tip first* and one *B: second CELL*, or *Park Only*. Pick the right alliance's OpMode. Each AUTO preselects TeleOp.
3. Place the robot on its start pose: touching the wall, wholly on your side, not in the LOADING ZONE, not touching a FLOWER.
4. Load exactly **4 POLLEN** (`inventory.preload_count`).
5. INIT, then START. Don't touch the robot in between.
6. Start TeleOp within **2 minutes** of AUTO so it inherits alliance, pose, HIVE state and inventory. Without an AUTO it defaults to RED. Press B (RED) or X (BLUE) on either gamepad during INIT.

9.2 Autonomous routines

OpMode	RED start	Plan
A: tip first	[58, 9, 90°], audience wall	4 POLLEN into CELL A (the field is set up to TIP at that point), around the HIVE to an angled CELL B pose, fires any leftovers if CELL B is up, parks low in the LOADING ZONE.
B: second CELL	[30, 132.5, 270°], rear wall	Waits at the CELL B pose for the partner's TIP, fires the pre-load, parks high. If CELL B never comes up, it detours to CELL A and empties there.
Park Only	[9, 75, 0°], alliance wall	LEAVE + PARK.

The robot never fires at a downward CELL. A **park guard** cancels the routine and parks when the time left reaches the estimated drive plus `park_margin_ms` (2.5 s), and never later than `park_reserve_ms` (3 s) before the end.

TeleOp cues

1:00 left	Two rumble blips, and "NECTAR→FLOWERS OK" next to the clock (G410/G426).
Park alert	From 0:20 it estimates the drive to park. Once the time left is that drive + 4 s, both pads rumble and flash red every second, the DS says "park now", and **PARK NOW** heads the telemetry until parked. F310 pads have no rumble or light. It never drives the robot itself: press G1 LT. With no clear route or a lost pose, it fires at 0:20.
Human NECTAR	One allowed per TIP of your HIVE, and all five after 1:00. The field timer is the authority.
"Shot" line	Names the gate that is blocking a feed.
"Pose assist"	Why a pose hold refused to move (it would hit the HIVE or a wall).

9.3 Gamepad cheat sheet

Gamepad 1: driver

Left stick	Translate (cubic, ≤ `max_speed`, through Casablanca). Deflecting past 0.2 cancels a pose hold.
Right stick X	Rotate. Releasing it holds the heading.
X	Toggle "face the upward CELL" (follows TIPS).
RT	Drive to and hold the shooting pose for the CELL that is up.
LT	Drive to and hold the park pose.
D←	FLOWER pose. *Off* (`flower_hold_enabled: false`).
D→	Cancel pose hold.
B / X in INIT	Choose RED / BLUE (only when no AUTO ran).

Gamepad 2: operator

A / B	Start / stop intake (it stops itself at a known 4).
D→ hold	Eject (reverse at `eject_power`).
RT hold	**Aim and shoot**: holds position, spins up, aims, and feeds when every gate passes.
Y hold	Turret aim only.
X hold	Rev the flywheel, no feed. X+Y = pre-spin and aim.
D↑	Fixed-speed shot at `constant_rpm`. Still gated, and blocked while uncalibrated.
LT	Cancel everything.
D↓ hold	Vision relocalize.
LB / Back	Record a missed TIP / undo the last TIP.
RB	POLLEN/NECTAR. No effect while NECTAR is disabled.
L3	Mark the robot empty (inventory 0).

10 Config reference

↑ contents

Every top-level section of `config.yaml`. **Now** is the repo value on 2026-11-05. For any key not listed, read its description and limits in `config-docs.yaml`.

MEASURE calibrate on your robot · PLACEHOLDER must replace · TUNED leave · POLICY deliberate choice · SWITCH on/off · FIELD game geometry · MODEL ONLY not live aim

10.1 `pedro` : drivetrain and path following

set by §7.2

Key	Now	Meaning	Status
`localization.x_pod_offset / y_pod_offset`	`0.883 / −5.463`	Pinpoint pod positions from the robot centre (in).	MEASURE
`heading.pid.p / i / d, heading.ks`	`4.27 / 0 / 0, 0`	Path-following heading feedback, P only.	MEASURE
`translation.gain_threshold`	`2.5 in`	Error at which each axis switches to its large-error gains.	MEASURE
`translation.forward.small/large_error_pid.p`	`0.114 / 0.309`	Forward position gains (I = D = 0). Marked provisional.	MEASURE
`translation.strafe.small/large_error_pid.p`	`0.144 / 0.389`	Strafe position gains (I = D = 0). Marked provisional.	MEASURE
`feedforward.coast_kv / brake_kv`	`0.01257 / 0.01069`	Foresight motion feedforward.	MEASURE
`braking.forward.linear / quadratic`	`0.0984 / 0.00158`	Stopping distance = `linear·v + quadratic·v·|v|` (inches). **Casablanca reuses these.**	MEASURE
`braking.strafe.linear / quadratic`	`0.0354 / 0.00214`	Same model, strafe axis.	MEASURE
`braking.heading.linear / quadratic`	`0.0533 / 0.00379`	Same model, heading (radians).	MEASURE
`motion.max_forward_velocity`	`79.97 in/s`	Top forward speed. Flagged "recheck".	MEASURE
`motion.max_strafe_velocity`	`64.77 in/s`	Top strafe speed.	MEASURE
`motion.natural_forward / strafe_deceleration`	`38.7 / 60.9 in/s²`	Coasting deceleration.	MEASURE
`motion.max_path_speed`	`0.9`	Fraction of top speed that autonomous paths use. Lower it for gentler autos.	POLICY

10.2 `field` · `hive` · `match`

from manual TU05 and the official CAD

Key	Now	Meaning	Status
`field.size_in`	`141.5`	Inside-wall width. The centre (70.75) is the BLUE rotation point. Re-measure if your field differs.	FIELD
`field.hive.*` (9 keys)	`see file`	Pivots 25.5 apart and 43.95 high. Up opening 15.69 out at 59.56 high, down opening 21.36 out at 38.17 high. Frame 49.46 × 38.95, bottom at 30.6.	FIELD
`field.loading_zone / garden / flower`	`see file`	Zone tape and FLOWER ring centres from the CAD.	FIELD
`hive.up_height_threshold_in`	`48.9`	A CELL seen above this height is "up". It is the midpoint of 59.6 and 38.2.	FIELD
`hive.confirm_frames_count`	`3`	Agreeing frames needed before an unexpected TIP is accepted.	POLICY
`hive.settle_ms`	`1500`	Feed lockout after any TIP, while the HIVE swings.	POLICY

Key	Now	Meaning	Status
`hive.max_observation_age_ms`	`400`	Ignore tag sightings older than this.	POLICY
`match.auto_ms / teleop_ms`	`30000 / 120000`	Period lengths.	FIELD
`match.flower_unlock_remaining_ms`	`60000`	The final-minute cue.	FIELD
`match.final_warning_remaining_ms`	`20000`	When the park alert starts watching.	POLICY

10.3 `inventory` : ball counting

Key	Now	Meaning	Status
`capacity_count / preload_count`	`4 / 4`	The G407 cap and the start load.	FIELD
`entry_sensor_enabled`	**`false`**	Set true only once a beam break named `entry_sensor_name` (`"intakeBeam"`) is wired. Without it, the count becomes unknown once the intake runs, and an unknown count never blocks the intake.	SWITCH
`entry_sensor_active_low`	`true`	The sensor reads false when blocked (most beam breaks). Flip it if counts look inverted.	SWITCH
`entry_debounce_ms`	`40`	How long the sensor must stay blocked to count one ball.	MEASURE

10.4 `turret`

set by §7.4

Key	Now	Meaning	Status
`enabled`	`true`	false: no turret, aim with the chassis heading.	SWITCH
`max_power_output`	`0.5`	Servo power cap, which sets the top turret speed.	POLICY
`servo_direction_inverted`	`false`	Flip if the turret drives *away* from its target. Plant Calibration detects this.	SWITCH
`ks_positive / ks_negative`	`0.0009 / 0.0663`	Breakaway power in each direction. It is very lopsided (see ③).	MEASURE
`orientation.logo / usb`	`LEFT / UP`	Mounting of `turnImu`, for Angle Verification only.	MEASURE
`travel.min_angle / max_angle`	`−26.5 / 26.5°`	Allowed angle relative to the chassis.	MEASURE
`stall.enabled`	`true`	Cuts power if commanded but not moving.	SWITCH
`stall.power_threshold / min_progress_deg / timeout_sec / retry_after_sec`	`0.12 / 0.75 / 0.35 / 1.0`	When a stall is declared and when power comes back. A retry of 0 latches the fault until restart.	POLICY
`runaway.enabled / divergence_deg`	`false / 8`	Cuts power if the error keeps growing (an inverted loop). Turn it on while debugging direction.	SWITCH
`pidf.p / i / d / f`	`0.02 / 0.008 / 0 / 0`	Turret feedback. The calibration and the proof want i = d = f = 0.	MEASURE
`kv · ff_min_rate_dps`	**`0`** · `3.0`	Chassis-motion feedforward (1/K), **0 = off**. Rates under 3°/s are ignored as noise.	MEASURE
`pivot_forward_in / pivot_left_in`	**`0 / 0`**	Turret axis offset from the robot centre. Aim and distances are measured from it.	MEASURE
`integral.band_deg / limit_power`	`30 / 0.08`	The I term only acts inside the band, and is clamped.	POLICY
`tolerance.max_drift`	`8 in`	Total miss allowed at the CELL. The angle is `asin(4/dist)`, clamped to [min_deg, max_deg].	POLICY
`tolerance.near_cutoff / near_val / min_deg / max_deg`	`1 / 1.5 / 0.5 / 4`	Tolerance shaping: a fixed 1.5° inside 1 in, otherwise 0.5–4°.	POLICY

Key	Now	Meaning	Status
`tolerance.exit_factor`	`1.5`	Settle hysteresis: once settled, it stays settled until the error exceeds tolerance × 1.5. This stops hunting.	POLICY
`analog_encoder.enabled` · `inverted`	`true` · `true`	Use the pot. `inverted` means the angle rises as the voltage falls. Never use it to fix the servo direction.	SWITCH
`analog_encoder.zero_voltage / degrees_per_volt`	`1.265 / 42.15`	Volts to degrees. Printed by Center Calibration.	MEASURE
`analog_encoder.min/max_voltage / full_scale_voltage`	`0.556 / 1.964 / 3.3`	The voltage range at the stops, and the pot's wrap voltage.	MEASURE

10.5 `vision`

set by §7.8

Key	Now	Meaning	Status
`enabled`	`true`	false: no camera. TIPS must then be entered by hand.	SWITCH
`continuous`	`true`	Stream all the time so every TIP is seen. false streams only while relocalizing.	SWITCH
`camera_name`	`"Webcam 1"`	Must match the DS configuration.	MEASURE
`decimation`	`3`	AprilTag speed versus range (1–6). Higher is faster with shorter range.	POLICY
`camera.forward_in / left_in / height_in`	`3.54 / 4.72 / 7.68`	Lens position on the robot.	PLACEHOLDER
`camera.yaw_deg / pitch_up_deg`	`0 / 50`	Lens direction. Pitch is positive (upward).	PLACEHOLDER
`relocalize.max_range_in / max_correction_in`	`72 / 18`	Furthest usable sighting, and the largest pose jump accepted.	POLICY

10.6 `sentinel` : geometric safety

Key	Now	Meaning	Status
`robot_width`	`18 in`	Side of the square footprint used for every zone check. **If it's wrong, safety is wrong.**	MEASURE
`rotation_lookahead_time`	`0.0685 s`	Predicts the footprint ahead of rotation. From Friction Calibration.	MEASURE
`protect_full_hive_footprint`	`true`	true: the whole HIVE base is no-drive. false protects only the two 2 in foot bars, for a robot built to pass under the HIVE (below 30.6 in, and clear of the inward-leaning legs, which are *not* modelled).	SWITCH
`frame_base_width_in`	`2.0`	Foot-bar width, used when the switch above is false.	FIELD
`center_line_margin_in / wall_contact_tolerance_in`	`0.25 / 0.5`	"Fully on your side" margin, and the distance that counts as touching a wall.	POLICY
`park_lane_margin_in`	`3.0`	HIVE clearance in the emergency park lane.	POLICY
`route_sample_step`	`0.005`	Path-check resolution. Smaller is more thorough.	POLICY
`park_turn_rate_dps / park_settle_per_waypoint_sec`	`180 / 0.25`	Conservative inputs for the park-time estimate.	POLICY
`geometry_epsilon_in`	`1e−9`	Floating-point tolerance. Don't touch it.	FIELD

10.7 `casablanca` : driver assist

set by §7.3

Key	Now	Meaning	Status
`enable_depth_protection / enable_side_protection`	`true / true`	HIVE protection on x and y. **Leave on** (G417). Turn off only to debug.	SWITCH

Key	Now	Meaning	Status
`repulsion.power / decel_safety_factor`	`0.12 / 0.7`	Push-back power inside a zone, and the braking-model margin (0.7 = 30% margin).	POLICY
`depth.slow_down / hard_stop` (same for `side`)	`8 / 2.5 in`	Where slowing starts, and where power toward the HIVE is cut. Keep depth and side equal.	POLICY
`lane_blend_distance`	`12 in`	Smooths slow-downs over this distance.	POLICY
`friction.x / y / rot`	`0.1041 / 0.1488 / 0.1041`	Power needed to break wheel stiction.	TUNED
`smoothing.teleop_enabled · auto_enabled`	`true · false`	Slew-limit TeleOp power. Slew-limit Pedro output in AUTO (off keeps native auto behavior).	SWITCH
`smoothing.forward.increase/decrease_time`	`0.22 / 0.55`	Seconds per unit of power change, per signed direction. 0 = unrestricted. From the Pod Contact Tuner.	MEASURE
`smoothing.strafe.increase/decrease_time`	`0.22 / 0.22`	Same, strafe axis.	MEASURE
`smoothing.turn.increase/decrease_time`	`0.088 / 0.088`	Same, turn axis.	MEASURE
`smoothing.max_loop_dt`	`0.05 s`	A stalled loop can't cause a big power step.	POLICY
`heading_lock.enabled`	`true`	Hold heading when the turn stick is released.	SWITCH
`heading_lock.pid.p / i / d · ks`	`5.82 / 0 / 0 · 0`	Heading-hold gains. From the Heading Lock Autotuner.	MEASURE
`heading_lock.ks_moving`	`0.0003`	Turn friction compensation while translating. From Friction Calibration.	MEASURE
`heading_lock.intent_threshold / moving_speed_threshold`	`0.05 / 10`	Stick level that counts as steering. Speed at which full `ks_moving` applies.	POLICY
`heading_lock.max_power / error_deadband_deg / settle_rate_dps`	`0.30 / 2.0 / 15`	Correction cap. No correction under 2°. Captures the heading once rotation drops below 15°/s.	POLICY
`heading_tuner.*` (≈20 keys)	`see file`	Autotuner setup: pose [20,72,90], ±10° steps, a P grid of 1–10 (4 values) and a D grid of 0–0.9 (4 values), plus tolerances and timeouts. `TunerPoseSafetyTest` checks the pose.	POLICY
`jump_tuner.*` (≈35 keys)	`see file`	Pod Contact Tuner setup: `trial_axis`, power 0.5, a 48 in lane, the fallback IMU (`imu` , RIGHT/UP), tilt and abort limits, and a slew search over 0.05–0.8 s with 12 candidates.	POLICY

10.8 `shooter`

set by §7.5 and §7.6

Key	Now	Meaning	Status
`use_ftc_pid`	`false`	**false**: this code's loop (feedforward + P + clamped I + voltage compensation). **true**: the hub firmware PIDF with `motor_pidf` , which oscillated with the old gains. `motor_pidf` stays on the hub until it is power-cycled.	SWITCH
`ks`	`0.1811`	Flywheel breakaway power.	MEASURE
`nominal_voltage`	`12.0 V`	The voltage the feedforward is expressed at. Commands scale by nominal ÷ measured. Set it to the loaded voltage seen while characterizing.	MEASURE
`max_voltage_compensation / voltage_refresh_sec`	`1.4 / 0.25`	Largest scale factor (covers down to about 8.6 V), and how often the voltage is re-read.	POLICY
`max_rpm`	`1500`	Ticks/s that target power 1.0 maps to.	MEASURE

Key	Now	Meaning	Status
`constant_rpm`	`1000`	The D-pad Up fixed shot, and the hold speed between volleys.	POLICY
`min_transfer_threshold / max_velocity_threshold`	`0.97 / 1.03`	The feed may *arm* only at 97–103% of target speed.	POLICY
`feed_release_threshold`	`0.80`	A running feed pauses if speed sags below 80%. Must be below `min_transfer_threshold`. Lower it if the intake stutters.	POLICY
`feed_intake_power / feed_transfer_power`	`1.0 / 1.0`	Intake powers while feeding the shooter.	POLICY
`ball_detection.dip_fraction / rebound_fraction`	`0.09 / 0.05`	A ball counts as a speed dip of more than 9% below reference. Rebound must be below dip.	TUNED
`ball_detection.baseline_alpha / refractory_ms`	`0.03 / 120`	Reference smoothing, and the minimum spacing between counted balls.	TUNED
`pidf.p / i / d / f`	`0.0010125 / 0.00050625 / 0 / 16.59`	Software loop: p = 2 kV, i = p / 2 s, f is the fitted feedforward.	MEASURE
`motor_pidf.p / i / d / f`	`2.204 / 0.2204 / 0 / 22.04`	Used only when `use_ftc_pid: true`. REV's recipe is p = 0.1 F, i = 0.1 p.	MEASURE
`integral.band_ticks / max_contribution`	`150 / 0.25`	The I term integrates only within 150 ticks/s of target, and adds at most 0.25 power.	TUNED
`kick.wheel_tau_sec / sensor_lag_sec`	**`0 / 0`**	From the step test. **0 disables the recovery kick** and the prediction.	MEASURE
`kick.kick_lag_margin_sec / kick_trigger_fraction / kick_speed_margin`	`0.0075 / 0.09 / 1.4`	Model margins. Keep the trigger fraction equal to `dip_fraction`.	TUNED
`ballistics.max_moving_speed_ips`	`20`	Shots are refused above this robot speed.	POLICY
`ballistics.min_valid_distance / max_valid_distance`	`24 / 72 in`	Valid shot range. The table's own range also applies.	POLICY
`ballistics.max_off_axis_deg`	`60°`	Refuses shots from too far to the side of the opening.	POLICY
`ballistics.preferred_shot_rpm / max_shot_rpm`	`800 / 1450`	**The table speed is clamped into this band** at runtime.	POLICY
`ballistics.rpm_update_deadband`	`25`	Minimum speed change before re-commanding the setpoint (each re-command resets the integrator).	POLICY
`ballistics.flight_time_base_sec / sec_per_inch`	`0.51 / 0.0049`	Time of flight, used only to lead moving shots.	PLACEHOLDER
`ballistics.lead_min_speed_ips / lead_bias_gain`	`10 / 1.0`	No lead below 10 in/s (noise). The lead multiplier.	POLICY
`ballistics.min/max_hood_servo_pos`	`0.85 / 0.0`	Servo values at the flattest and steepest hood positions. Reversed on purpose. Calibration clamps hood moves to this range.	MEASURE
`ballistics.min/max_hood_angle_deg`	`15 / 65°`	Physical hood angles at those servo values. Used for the angle display.	MODEL ONLY
`ballistics.v0 / k / magnus_l`	`250 / 0.0012 / 0.015`	Flight-model exit speed, drag and spin lift. The live aim comes from the table.	MODEL ONLY
`ballistics.launch_height_inches / g_inches_per_sec2`	`15 / 386.088`	Release height and gravity, for the flight model.	MODEL ONLY
`ballistics.v0_reference_rpm / v0_margin_fraction`	`1000 / 0.12`	Speed↔exit-speed conversion (reported in the solution), and long-shot headroom in the model.	MODEL ONLY

Key	Now	Meaning	Status
`shot_table.calibrated`	`false`	**While false, nothing ever feeds.** Set true after Ballistics Calibration.	SWITCH
`shot_table.nectar_enabled`	`false`	true allows selecting the NECTAR table. This robot can't handle NECTAR.	SWITCH
`shot_table.pollen_points`	`5 rows, 24–72`	Flat `distance, hood_servo, ticks/s` triples with strictly increasing distances. Currently guesses from a launch model.	PLACEHOLDER
`shot_table.nectar_points`	`5 rows`	Same format, for NECTAR. Unused.	PLACEHOLDER

10.9 `auto` : autonomous behavior

Key	Now	Meaning	Status
`shot_time_points`	`24:2200, 72:3000`	`distance, window_ms` pairs from the Shot Timing Tuner. A volley ends when the ball count is reached *or* the window expires (the jam safety net).	PLACEHOLDER
`shoot_wait_ms`	`2500`	Fallback window, used only if the table above is empty.	POLICY
`balls_per_volley_count`	`4`	Balls expected per volley.	POLICY
`tip_wait_ms`	`2500`	How long to wait for the target CELL to be seen up before giving up on it.	POLICY
`collect_dwell_ms`	`1200`	Time at a FLOWER with the intake on. Unused here.	POLICY
`park_margin_ms / park_reserve_ms / park_update_ms`	`2500 / 3000 / 250`	Park when the time left ≤ drive estimate + margin, or at the reserve time at the latest. Re-estimated every 250 ms.	POLICY
`transfer_power`	`0.12`	Transfer power while collecting in AUTO.	POLICY
`cell_a_collect_flower`	`false`	The CELL A routine refills at the side-wall FLOWER. Off: no mechanism.	SWITCH
`cell_a_second_volley`	`true`	Drive around the HIVE and fire leftovers into CELL B once it is up.	SWITCH
`cell_b_collect_flower`	`false`	The CELL B routine refills at the rear-wall FLOWER. Off.	SWITCH

10.10 `teleop` and poses

RED values; BLUE = `m`

Key	Now	Meaning	Status
`max_speed`	`1`	Driver translation scale (0–1). Lower it for a new driver.	POLICY
`field_centric`	`false`	true: the stick points in a fixed field direction. false: robot-relative.	SWITCH
`field_centric_offset_deg`	`0`	Field heading for "stick forward" when field-centric (90 = +y, away from the audience).	POLICY
`intake_power / transfer_power`	`1.0 / 0.1`	Operator intake (G2 A).	POLICY
`manual_rev_power / eject_power`	`0.6 / -0.6`	Operator rev (G2 X) and eject (G2 D→, negative = reverse).	POLICY
`flower_hold_enabled`	`false`	Driver D← FLOWER pose. Off: no pickup.	SWITCH
`rumble_ms`	`600`	Rumble length for match cues.	POLICY
`park_alert_margin_ms / _update_ms / _repeat_ms`	`4000 / 250 / 1000`	Park alert timing.	POLICY
`poses.red.start / park`	`[58,9,90] / [16,99,90]`	TeleOp start (if no AUTO ran) and park hold.	MEASURE
`poses.red.shoot_a / shoot_b`	`[58,27,90] / [58,114,270]`	Shooting poses facing CELL A and CELL B (G1 RT).	MEASURE

Key	Now	Meaning	Status
`poses.red.flower`	`[16,47.36,180]`	FLOWER pickup pose (disabled).	POLICY
`auto_poses.cell_a.red.*`	`6 poses`	The *A: tip first* waypoints: start [58,9,90] · score_a [58,27,90] · flower [16,47.36,180] · score_b_cp [16,100,0] · score_b [36,106,318.36] · park [16,99,90]. `score_b_cp` is a Bezier control point that keeps the swept path clear of the HIVE.	MEASURE
`auto_poses.cell_b.red.*`	`7 poses`	The *B: second CELL* waypoints: start [30,132.5,270] · score_b_cp [30,114,0] · score_b [58,114,270] · flower [47.36,125.5,90] · alt_via [30,104,0] · alt_a [32,33,40.31] · park [16,119.5,270]. `alt_via` and `alt_a` are the fallback route to CELL A.	MEASURE
`auto_poses.park_only.red.*`	`2 poses`	LEAVE + PARK: start [9,75,0] · park [16,99,0].	MEASURE

10.11 Every switch in one table

Every on/off feature flag in the config, its current setting, and when to flip it.

Switch	Now	Flip it when…
`shooter.shot_table.calibrated`	`false`	…Ballistics Calibration rows are pasted. **Nothing feeds until it's true.**
`shooter.shot_table.nectar_enabled`	`false`	…a NECTAR mechanism exists and `nectar_points` is calibrated.
`shooter.use_ftc_pid`	`false`	…you deliberately want the hub's firmware PIDF (`motor_pidf`). Not recommended.
`inventory.entry_sensor_enabled`	`false`	…an intake beam break named `intakeBeam` is wired.
`inventory.entry_sensor_active_low`	`true`	…the ball count goes up when the beam is *clear*.
`turret.enabled`	`true`	…the robot has no turret (aim with the chassis instead).
`turret.servo_direction_inverted`	`false`	…the turret drives away from its target (Plant Calibration says INVERTED).
`turret.analog_encoder.inverted`	`true`	…the measured angle moves opposite the real turret. Never use it to fix the servo direction.
`turret.analog_encoder.enabled`	`true`	…never, on this robot. Aiming needs the encoder.
`turret.stall.enabled`	`true`	…never, outside a bench test.
`turret.runaway.enabled`	`false`	…you are debugging turret direction (set it true for safety).
`vision.enabled`	`true`	…no camera is fitted. TIPS are then recorded by hand (G2 LB).
`vision.continuous`	`true`	…you need CPU and only relocalize. You will miss TIPS.
`sentinel.protect_full_hive_footprint`	`true`	…the robot is built to drive under the HIVE (low, and clear of the legs).
`casablanca.enable_depth_protection` / `…side…`	`true`	…never, in a match. Only for debugging, and always both together.
`casablanca.smoothing.teleop_enabled`	`true`	…comparing feel without the slew limiter.
`casablanca.smoothing.auto_enabled`	`false`	…pods lift during autonomous paths.
`casablanca.heading_lock.enabled`	`true`	…drivers prefer free rotation when the stick is released.
`auto.cell_a_collect_flower` / `cell_b_collect_flower`	`false`	…a FLOWER pickup mechanism exists.
`auto.cell_a_second_volley`	`true`	…the drive around the HIVE costs more than the leftover POLLEN is worth.
`teleop.flower_hold_enabled`	`false`	…a FLOWER pickup mechanism exists.
`teleop.field_centric`	`false`	…drivers want field-relative sticks (also set the offset).
`shooter.kick.wheel_tau_sec` (0 = off)	`0`	…Shooter Characterization's step test passes (paste its value).
`turret.kv` (0 = off)	`0`	…Plant Calibration gives K (paste 1/K).

11 Troubleshooting

↑ contents

Shooting

Symptom	Cause and fix
Never feeds a ball	`calibrated: false`, or a gate is failing. The TeleOp "Shot" line names the gate. Also check the distance is inside the table, speed is under 20 in/s, and the robot is within 60° of the opening axis.
Wheel sits just under target	Feedforward is off. Re-run Characterization on a loaded battery, and check `nominal_voltage`.
Overshoot after each ball	The I term is too strong. Lower `pidf.i` or `integral.max_contribution`, or enable the kick.
Balls not counted	Adjust `dip_fraction` (in the Shot Timing Tuner).
Short-range shots go hot	Table speeds below 800 are clamped up. Lower `preferred_shot_rpm`.
Intake stutters while feeding	Lower `feed_release_threshold` (keep it below 0.97).
Misses consistently to one side	Pivot offset, odometry drift (relocalize with G2 D↓), or pod offsets.

Turret

Symptom	Cause and fix
Drives away from target	Flip `servo_direction_inverted`. Turn on `runaway` while debugging.
Hunts or buzzes	Lower `pidf.p`, set `d: 0`, raise `exit_factor`, re-run Plant Calibration.
Creeps at zero power	Servo neutral is off. Re-centre it, then re-measure kS.
Lags a turning robot	`kv` is 0. Set it to 1/K.
Stall faults	Binding, or `stall.power_threshold` / `min_progress_deg` too tight for encoder noise.

Driving and vision

Symptom	Cause and fix
Odometry drifts or jumps	A pod is lifting (Pod Contact Tuner), or pod offsets or directions are wrong (Pinpoint Tuner).
Robot reaches the HIVE	`robot_width`, braking coefficients or `hard_stop` is wrong. Re-run Casablanca Test.
Pose hold refuses	See "Pose assist". The direct path hits the HIVE or a wall, so drive to a clear approach first.
HIVE never flips	Camera mount values, name, tilt, `vision.enabled`. Record TIPS with G2 LB meanwhile.
Relocalize rejected	HIVE not settled, no fresh frame, over 72 in away, or a jump of more than 18 in.

Build and deploy

Symptom	Cause and fix
Pushed config ignored	Re-INIT. Read the DS banner (season mismatch). Check that `pushConfig` succeeded.
`pushConfig` fails	Read the failing test name: route safety, sanity, or match window.
"red_ without blue_"	Add the other alliance's key.
Build rejects a value	Outside `min` / `max` in `config-docs.yaml`, or the key has no description.
`adb devices` empty	Use an A-to-C cable, check macOS Local Network permission, join the robot Wi-Fi.
Gradle silent	Frozen daemon. Use `--no-daemon`.
Max Velocity Test acts oddly after a firmware-PID run	`motor_pidf` persists on the hub. Power-cycle it.
Intake passes 4 balls	No sensor. Wire one, or the operator presses G2 B.

12 Known gaps and further reading

↑ contents

Not built, or limited

- **No FLOWER mechanism.** All three flower switches are off, so each auto fires only its 4 pre-loads.
- **No NECTAR mechanism.** Every shot uses the POLLEN table.
- **No color sensor.** The operator ejects opponent NECTAR (G408).
- **Continuous feeding** while every gate passes. A TIP detected mid-volley stops further feeding, but balls already in the air can't be recalled.
- **Hood settling is not measured**: the servo readback is just the command. A clamped speed is not re-validated against the table.
- **Moving shots** use first-order lead only, with a conservative 20 in/s limit.
- **Hood range.** A launch-model estimate wants a 64–78° lob, and the hood tops out at 65°. Confirm the real mechanism scores at 24–48 in.
- **Calibration reach.** Both calibration OpModes stop at 48 in (the ray ends around 52 in), so longer shots need a different ray.
- **Proof scope.** The proofs cover the control logic for a stated plant model and the conditions `conditions.py` checks. They don't show the hardware matches the model. §8.3 does.
- **Unused hardware.** The `gate` servo appears in `hwmap.yaml`, but no code drives it.

Further reading in the repo

`AGENTS.md`	Coding standards and architecture rules, the contract for editing Java.
`BIOBUZZ-SEASON.md`	Rules mapped to code, the field geometry table, the bring-up checklist.
`teleop_controls.md`	The authoritative TeleOp controls.
`MIGRATION-PEDRO3.md`	Pedro 3 decisions and the AutoTune bring-up.
`robot/config/config.yaml`	Every value.
`robot/config/config-docs.yaml`	Every description and limit.
`teamcode/hwmap.yaml`	Hub ports.
`calibration-data/`	Past calibration output (last season's, so don't reuse it).

✓ **Done means:** every MEASURE and PLACEHOLDER in §10 is replaced, `calibrated: true`, `verifyBuild` passes, the final config is in the APK (after `resetConfig`), and all 13 acceptance tests pass.